Server architecture

A server (or a node) is a bundle of server applications and databases.

Applications:

1. **Hub** — launches and stops the other server applications and routes messages between them.
2. **Shard** — authorization, purchases, mail, ship equipment, and other.
3. **LoadBalancer** — balance between separate nodes.
4. **MatchMaker** — players/teams matchmaking, creates game sessions, adds players to the created games.
5. **DedicatedServer** — game rooms server. Launches DedicatedServer when launching a battle.
6. **GameRewardDispencer** — issues rewards after battles/sessions.
7. **Authenticator** — user authorization/online condition service.
8. **ChatServer** — used for communication between players.
9. **ConfigServer** — updates configuration of separate services.
10. **Universe** — responsible for the global condition of the “universe”/zone maps and clan conflicts.
11. **QuestServer** — controls player quests: condition, progress, completion, synchronization with zones/scripts and rewards.
12. **ZoneCoordinator** — coordinates ZoneInstance: tracks active zones, directs players to the zones.
13. **ZoneInstance** — same as DedicatedServer, but for the open world.
14. **LobbyServer** — manages the lobby before the game starts: rooms, player lineups, readiness, and match start settings.
15. **AchievementServer** — stores and updates players’ achievement progress and issues rewards for achievements.
16. **LeaderboardServer** — service for leaderboards.
17. **ClanServer** — clan logic service: clan data, membership, clan-related requests, and associated operations.

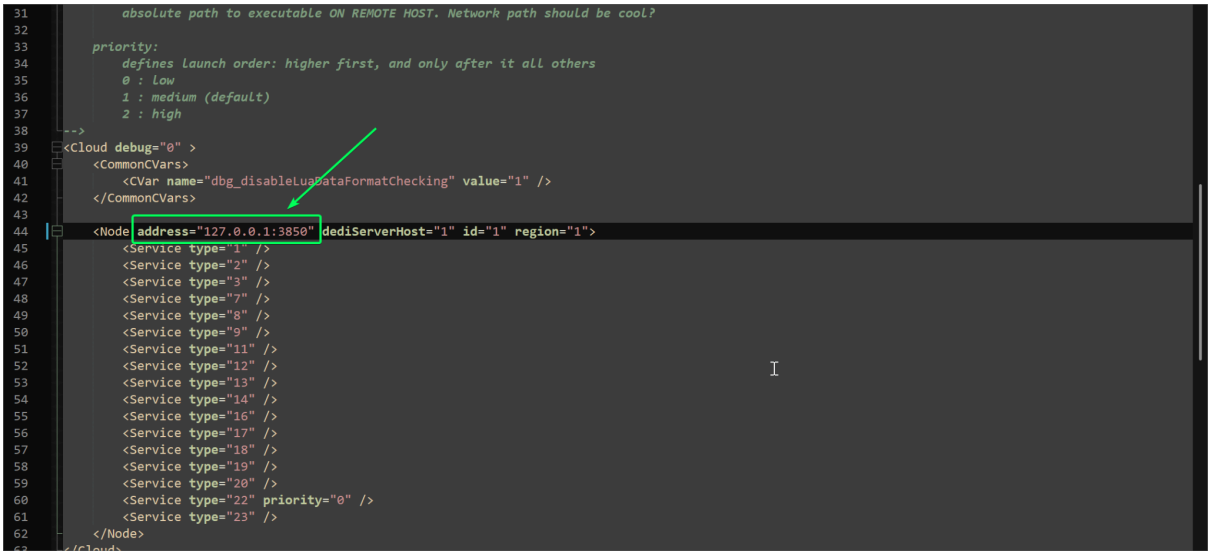
18. **Tournament** — tournament management service: launches tournaments according to the schedule, gathers registered players into teams, creates tournament matches, tracks victories by round, and sends players updates on the tournament's progress.
19. **ScriptServer** — service for running scripts.
20. **AdventureServer** — manages adventure scenarios/events: adventure stages, conditions, activity statuses, and their updates.

Databases:

1. **MongoDB** — main database.
2. **Redis** — database for leaderboards. You will need to download and install this database independently if you want to enable leaderboards.

Server settings

The server address is configured in the **cloud.xml** configuration file. The default address is **127.0.0.1:3850**. In this case, you can connect only from your local PC. If you need to connect from remote PCs, you must enter the address of the PC running the server.



Please note that even if the 127.0.0.1 address is specified, this doesn't guarantee that remote PCs will be unable to connect; the server accepts

connections from any address but provides clients with the connection address specified in the cloud.xml configuration. In other words, regular clients will not be able to connect, but an attacker could modify a client and connect to the server if they know your computer's IP address and can use that address to connect to your PC (a public IP or one from your local network). You must take additional measures independently to prevent unauthorized users from connecting to your server.

Connection to the server

There are 3 ways to launch the server: automatic, manual launch of databases and server applications and using a special button on the game's login screen.

Automatic launch

When the game client is launched via Steam, the -launchLocalServer parameter is transferred to the game client, indicating that the databases and the server must be started. The local_server.xml configuration file, which sets up the automatic launch of the databases and the server, is located next to game.exe. The launch takes place in the following order:

- MongoDB.
- Waiting for the TCP port of MongoDB to open.
- Redis, if it is specified in the configuration.
- Waiting for the TCP port of Redis to open.
- Hub.
- Checking that the Hub process has not ended within the time specified in the configuration.

If the launch of one of the processes fails, the client attempts to shut down the processes that are already running. If a process does not accept the shutdown command or does not manage to shut down within the allotted time, it may continue to run. In this case, the client continues to operate.

When the client is closed, processes are shut down in reverse order. First, a request for a soft shutdown is sent; the client then waits for the process for no longer than the time specified in the configuration. The processes cannot be forcibly shut down.

In the local_server.xml configuration file, you can enable or disable the launch of individual processes, enable their soft shutdown when the client is closed, and configure the waiting timeouts for launch and shutdown.

Manual launch (recommended)

1. Launch MongoDB using the **pc\launch_mongodb.bat** file. Wait until the database is fully launched.
2. Launch Redis if you have it installed and wait for its launch.
3. Launch the server applications using the **launch_server.bat** file.

Using the game client

You can launch the server by using the “Launch server” button on the login screen. This button executes the **launch_server_and_db.bat** file. The file launches the MongoDB database, waits for 3 seconds, and launches Hub, which launches the rest of the server applications. In this case, there is no guarantee that the databases will load before the server applications start running, which could cause the server applications to malfunction. If you have Redis installed, you can add a command to launch it to the launch_server_and_db.bat file.

Running server applications in windowed and windowless modes.

By default, Hub runs server applications in windowless mode. If necessary, you can change the launch mode so that applications run in a window, for example, if you need access to each application’s console. To do this, set

the hub_detachChildProcesses to 0 (set hub_detachChildProcesses 0) in the “pc\cloud\data\hub.cfg” configuration file.

Stopping the server

The server can only be shut down properly if the following sequence of steps is followed:

1. The server is switched to maintenance mode. To do this, run the /enterMaintenanceMode console command in the Hub, specifying the number of minutes after which the server will enter this mode.
2. Wait until all game clients have disconnected from the server.
3. Close Hub using the /quit console command.
4. Close the MongoDB database (it’s recommended to press Ctrl+C in the application window and confirm that you want to close it when prompted).
5. Close the Redis database if you have it installed.

Connecting a client to the server

If you have a local server running, select “localhost” on the login screen (this is selected by default).
If you want to connect to a remote server, select “server” from the drop-down list on the login screen; fields for entering the remote server’s address and port will appear below.
Always connect only to servers you trust, as the game client may contain vulnerabilities that could allow attackers to compromise your PC.
After selecting a server, enter your username and password, then click the “Login” button. If there is no account with the specified username in the database, a new one will be created.
Next time you log in to your account with the same username, you’ll need to enter the password you used when creating the account.

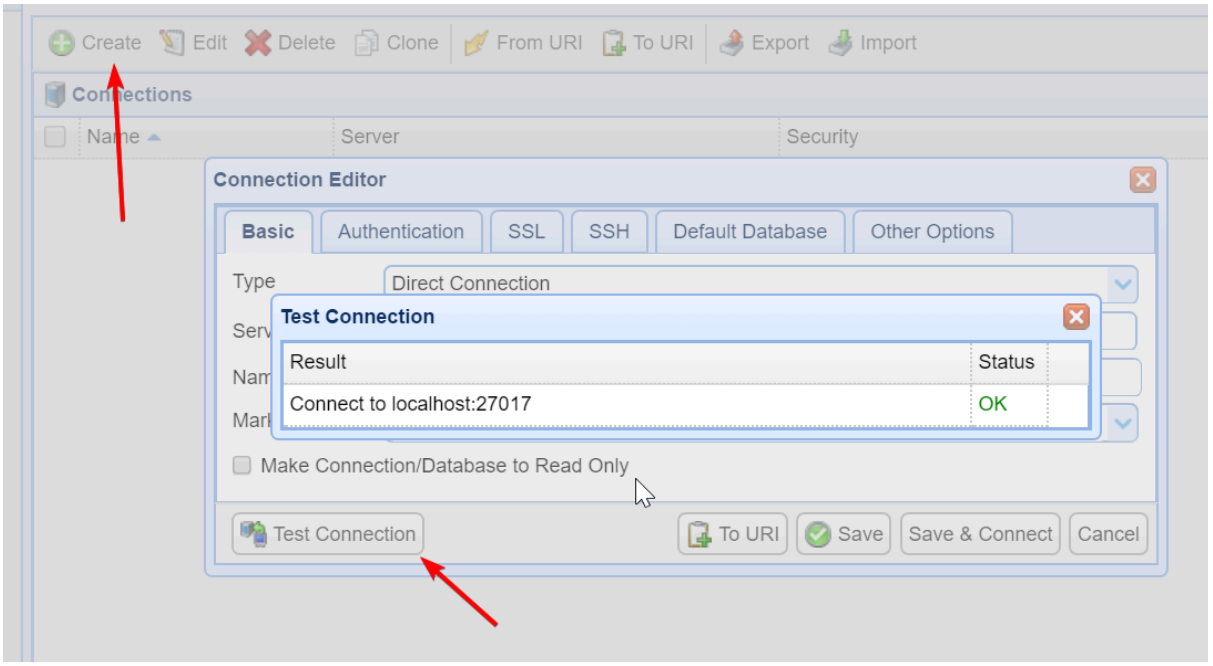
If you want to prevent the creation of new accounts, you can set the value of the variable in the Shard configuration to **cvars.auth_allowInternalAutoCreate = 0**

Working with databases

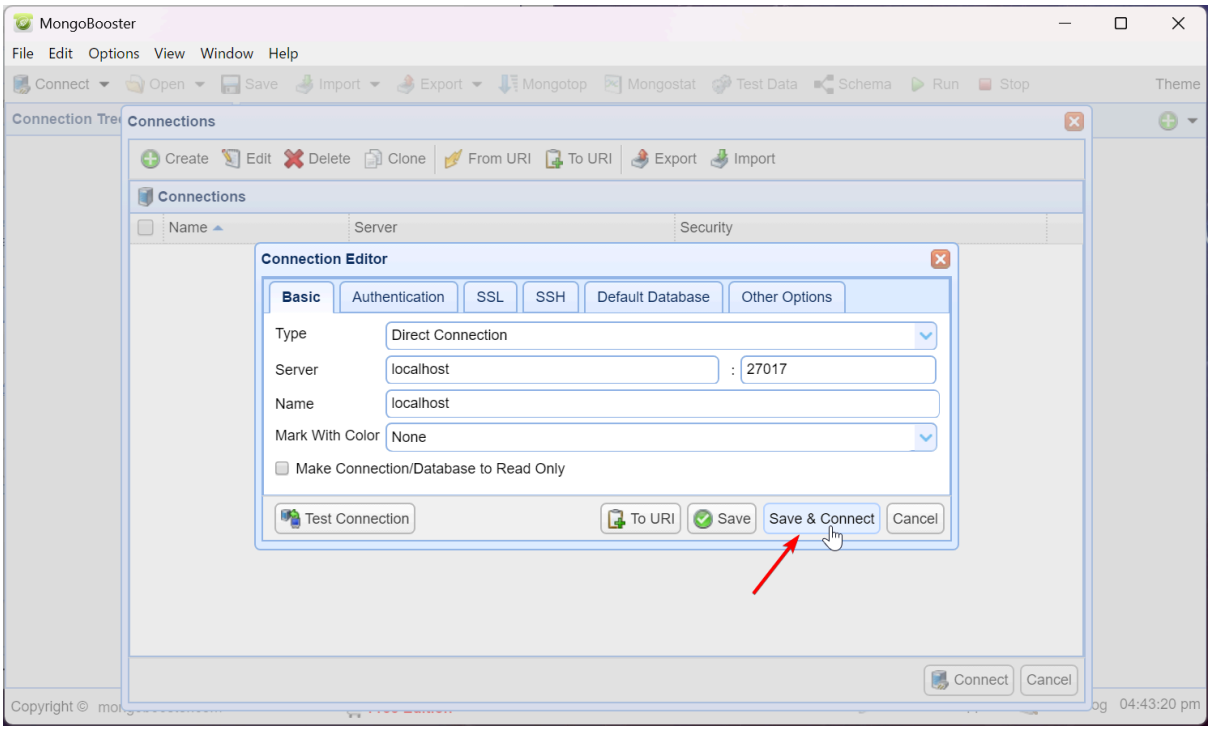
MongoDB

Before working with the database, you need to make sure that your server and the game itself are not launched; otherwise, the changes in the database may be cancelled.

To work with the database, you can install **Mongobooster** or other similar tools that support **MongoDB**. Let's take a look at **Mongobooster** as an example.
We need to connect to the database. To do this, launch the database using the **launch_mongodb.bat** file.
Check in Mongobooster to make sure we have a connection to the database. To do this, click Create -> Test Connection.



Next, connect to your database by pressing “Save&Connect”.



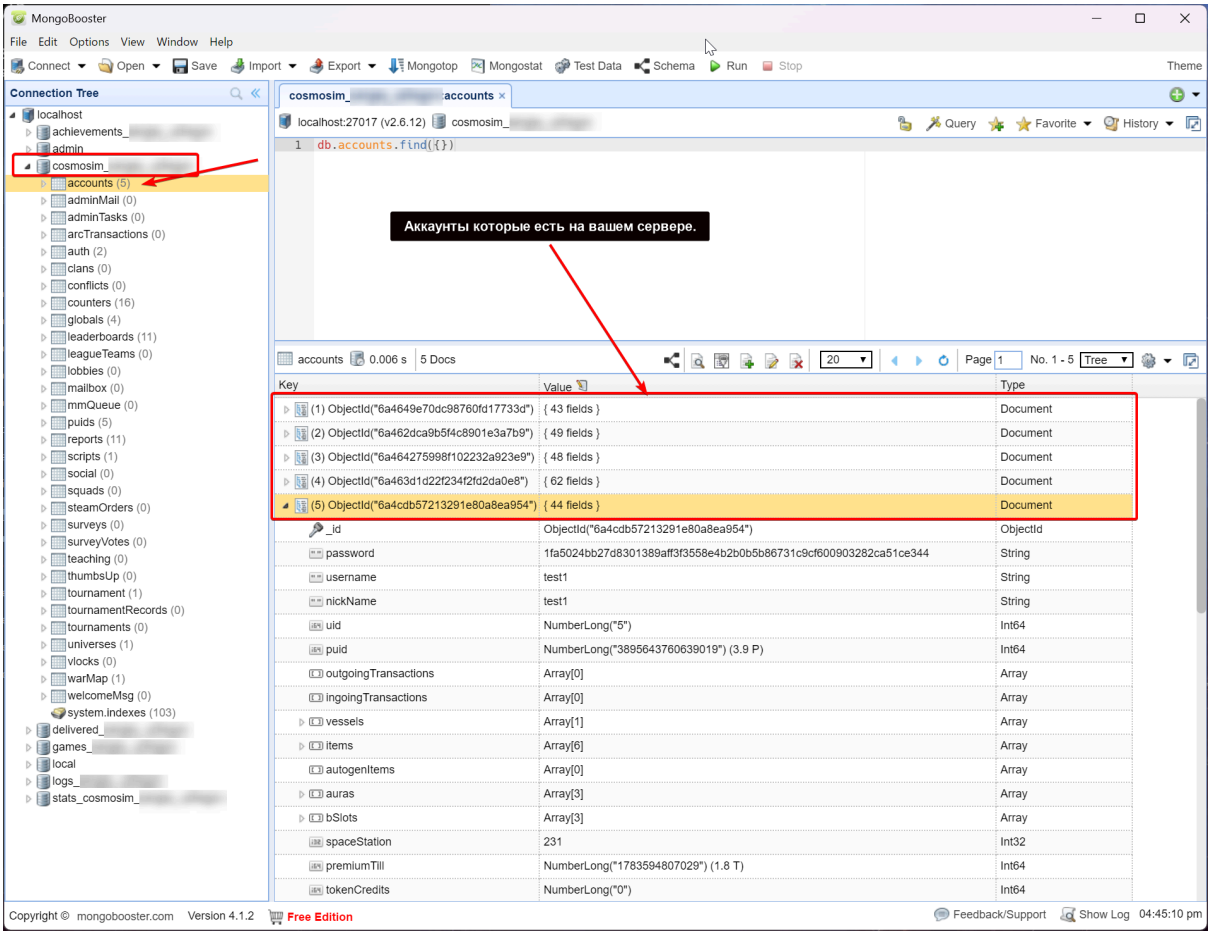
Once you are connected, the database interface will appear.

The database tree is displayed on the left, and on the right, the stored data will be displayed when you select one of the items.

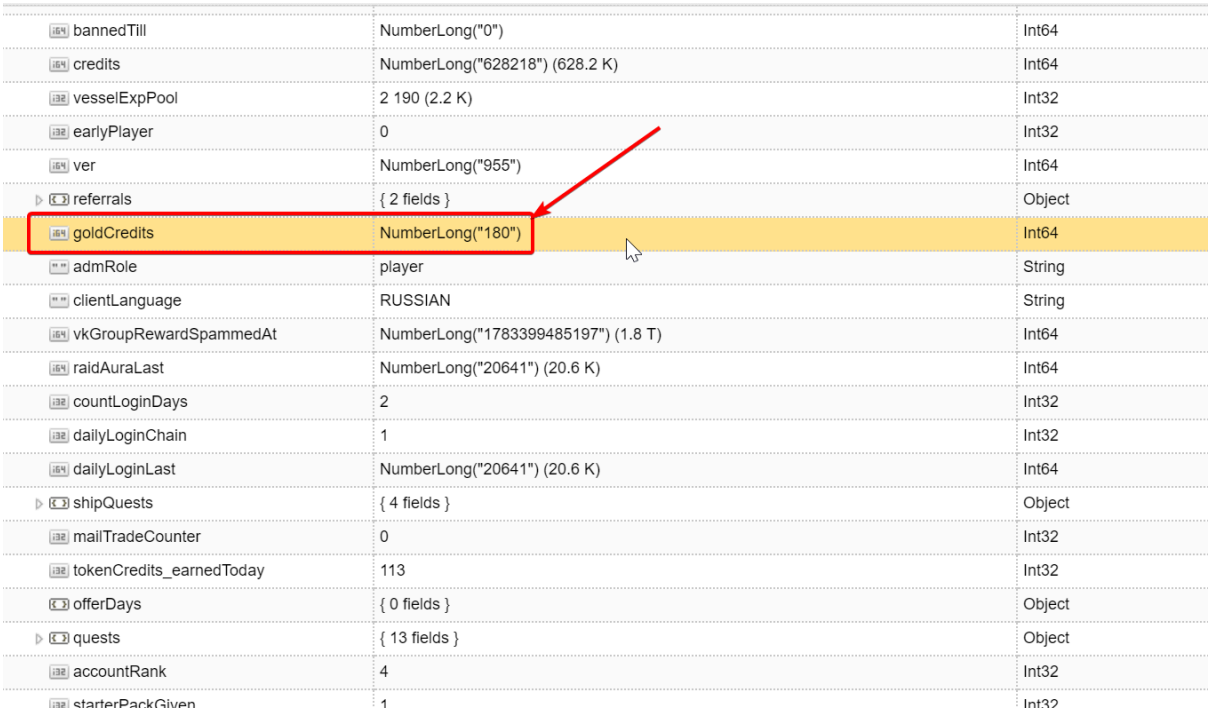
Let's take a look at an example with accounts on the server.

Select **cosmosim_%YOUR_LOCAL_NAME_WINDOWS%** and select **accounts** in the opened branch. All accounts that have been registered on your server will be displayed here, including all of your own accounts.

As you can see, the password here is encrypted, so you can’t see it here if you forgot it.



In the account data, you can find various details showing what a player has in their account; for example, you can “adjust” the Galactic Credits balance on your account. To do this, locate the **“goldCredits”** parameter in the database and set it to the desired number.



bannedTill	NumberLong("0")	Int64
credits	NumberLong("628218") (628.2 K)	Int64
vesselExpPool	2 190 (2.2 K)	Int32
earlyPlayer	0	Int32
ver	NumberLong("955")	Int64
referrals	{ 2 fields }	Object
goldCredits	NumberLong("180")	Int64
admRole	player	String
clientLanguage	RUSSIAN	String
vkGroupRewardSpammedAt	NumberLong("1783399485197") (1.8 T)	Int64
raidAuraLast	NumberLong("20641") (20.6 K)	Int64
countLoginDays	2	Int32
dailyLoginChain	1	Int32
dailyLoginLast	NumberLong("20641") (20.6 K)	Int64
shipQuests	{ 4 fields }	Object
mailTradeCounter	0	Int32
tokenCredits_earnedToday	113	Int32
offerDays	{ 0 fields }	Object
quests	{ 13 fields }	Object
accountRank	4	Int32
starterPackGiven	1	Int32

To apply the changes to the database, click the "**Run**" button on the top toolbar.

Removing the players' data from the database

If you want to play from the start, you can create a new account by entering a username in the login window that isn't already in the database. You can also delete all users and all data that the server has stored in the database by deleting the folder where the MongoDB data is stored: **%LOCALAPPDATA%\Targem\StarConflict\cloud\mongodb\data**. If you have a Redis database (or a compatible one) installed and the server was connected to it, you can also delete its data.

Redis

In order to enable leaderboards, you can download the Redis database or its counterparts, which support your operating system. Let's look at the Redis DBMS (database management system) as an example. First, we need to launch **redis-server.exe**.

In the **common.cfg** game configuration file, we need to uncomment the **set db_redisHost "127.0.0.1:6379|connpool=5"** line. This is necessary for the game to connect to the Redis database. If you are using a different database, you will need to modify the configuration to match your DBMS.
To have Redis start automatically when you click “Start Server,” you need to configure it to launch in the **launch_server_and_db.bat** file.
Example: **start "" "C:\Work\build\Redis-x64-5.0.14.1\redis-server.exe"**
Here’s how to properly shut down Redis:
Run redis-cli.exe and execute the SHUTDOWN command with the SAVE parameter.

Modifying the game’s databases

The game data is being stored in the “**pc\data**” folder.
Some of the game data files can be used by the client and then the server at the same time.

Certain server applications can read data from the same lua files as the client, such as DedicatedServer. However, most server applications read data from XML files that are generated based on the data in lua.

The game data for the server is stored in the “**pc\cloud\data**” folder. If you need to change the game data that is simultaneously stored in the lua and xml files, you need to change the data in the lua files first. Then you need to launch the “**pc\cloud\prepare_shared.bat**” file, which will generate the necessary xml files. Whenever you make any changes to the data in the lua files, simply run the “**pc\cloud\prepare_shared.bat**” file.

Matchmaker settings

Most of the settings can be found in the "**pc\cloud\data\default_platform_config.lua**" file.

For each server application, there is a corresponding Lua function; for example, "ConfigureShard" for Shard or "ConfigureMatchmaker" for Matchmaker. The settings are configured within those functions.

The **"pc\cloud\data"** directory also contains configuration files for each application, such as shard.cfg or matchmaker.cfg, as well as a general configuration file, common.cfg, where you can specify values for variables that should apply to multiple server applications at once.

Here are the localized names of the game modes used in the matchmaker settings:

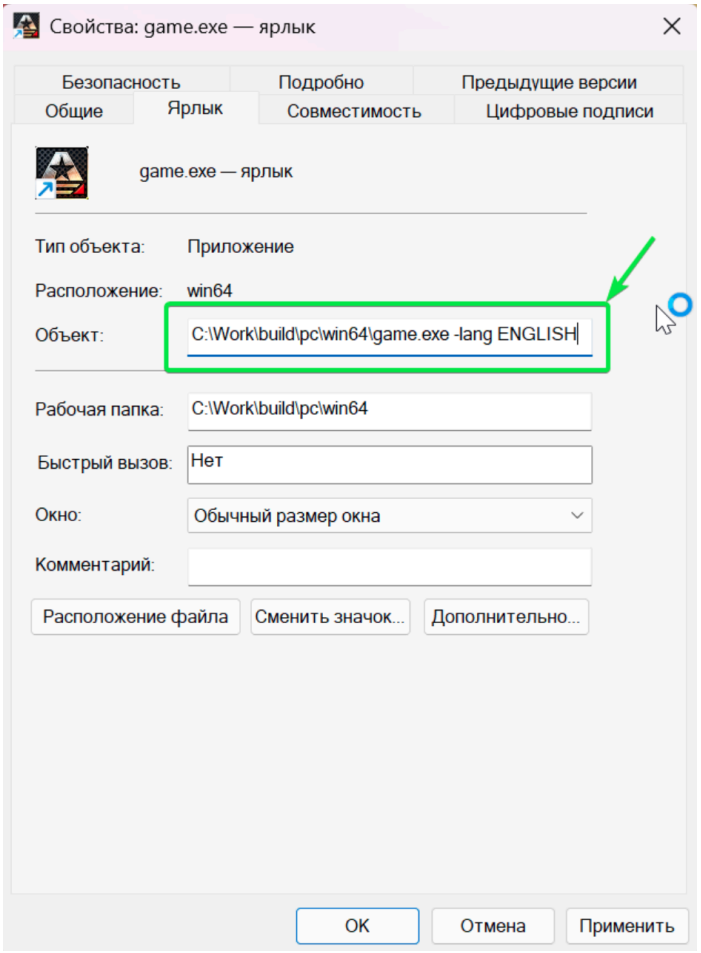
- "**april_control**" — "Paper conflict"
- "**ARENA**" — "Portals"
- "**atdm**" — "Paper conflict"
- "**BRAWL**" — "Brawls"
- "**capture_the_base**" — "Beacon capture"
- "**capture_the_base_2**" — "Four lives"
- "**CloseDistance**" — "Close contact"
- "**CLOSE_DISTANCE**" — "Close contact"
- "**COOP_VS_AI**" — "Skirmish"
- "**CUSTOM_GAME**" — "Custom battle"
- "**football**" — "Spaceball"
- "**gtdm**" — "Survival"
- "**hunger_games**" — "Last man standing"
- "**one_against_all**" — "Beetle in the Anthill"
- "**OPEN_WORLD_MAP**" — "Map"

- "**OPEN_WORLD_START**" — “Undock”
- "**PVE_MISSION**" — “Pick Scenario”
- "**PVE_MISSIONS**" — “Missions”
- "**PVE_RANDOM**" — “RANDOM MISSION”
- "**PVP**" — “PvP Arena”
- "**QUESTS**" — “Pick Scenario”
- "**RAID**" — “Special Operation”
- "**RANDOM_BFS**" — “Conquest”
- "**sentinel**" — “Combat recon”
- "**three_teams_dm**" — “Orion's belt”
- "**TOURNAMENT**" — “Tournament”
- "**zombie**" — “Curse of the Leviathan”

The description of the matchmaker settings can be found in the **matchmaking.md** file.

Localization settings

To change the in-game language you can add the following to the game’s desktop shortcut:
-lang RUSSIAN
-lang ENGLISH
-lang FRENCH
-lang GERMAN
-lang SPANISH
-lang POLISH
-lang PORTUGUESE
-lang HUNGARIAN
-lang CZECH
-lang UKRAINIAN
-lang CHINESE



You can also change the game language in the user_config.xml file, which can be found here: **%LOCALAPPDATA%\Targem\StarConflict**

Logs

The server application logs are stored in the **“%LOCALAPPDATA%\Targem\StarConflict\cloud\Hub\logs”** folder.